

Anonymity Preserving, Decoupled Distributed Authentication for Multihop IoT Networks

Sreeparna Das, Anup Kulkarni, Manas Khatua

Computer Science and Engineering

Indian Institute of Technology Guwahati

{d.sreeparna, anup.kulkarni, manaskhatua}@iitg.ac.in

Abstract—The Internet of Things (IoT) demands lightweight and secure authentication for resource constrained multihop networks, while also protecting the privacy of individual devices. Based on our analysis of the lightweight decoupled distributed authentication scheme of Das [1], we identify two critical gaps. First, the real device identity is exposed on the open channel in every authentication session, enabling adversarial cross-session tracking. Second, packet loss in the session key establishment phase desynchronises the device and server states, forcing full re-enrolment. We propose a security improved and anonymity preserving enhanced version of Das [1], addressing both identified gaps while preserving the core features of the original scheme, including PUF-based secret derivation, hash-based accumulator membership verification, decoupled authentication server selection, and the lightweight hash-XOR computational structure. Device anonymity is preserved through per-session pseudonym rotation, where a rotating pseudonym replaces the real device identity in all open-channel communications, preventing long-term tracking. A dual-state synchronization mechanism maintained at both the authentication server and the gateway enables seamless recovery from single packet-loss events without re-enrolment. Security is validated formally (ProVerif) and informally. COOJA/Contiki-NG simulations confirm that the proposed scheme achieves a per-device energy of 52.21 mJ and a communication cost of 1376 bits, reducing total energy by 42.8 % over LAACA [2] and 34.6 % over Zhou [3], while maintaining the lowest overhead among all compared schemes.

Index Terms—Lightweight, Authentication, Anonymity, Pseudonym, Physically Unclonable Function (PUF), Dual-State Storage, Proverif, COOJA.

I. INTRODUCTION

The Internet of Things (IoT) connects millions of smart devices that collect and transmit data to centralized platforms [4] such as gateways, often through multihop paths [5]. Industrial and outdoor deployments expose devices to a range of adversarial threats, making authentication a prerequisite for any communication session. Because IoT nodes are battery-powered and resource-constrained, authentication schemes must be *lightweight* as well as *secure* [6].

Centralized schemes route all authentication traffic through a gateway, imposing excessive relay load on intermediate nodes in a multihop topology [7] [8] [9]. Distributed schemes distribute this responsibility. However, a plain parent-node scheme overloads whichever node a set of new devices happened to join under, while all other capable nodes remain idle [10].

Das [1] addressed this skewed-load problem by proposing a lightweight decoupled distributed authentication scheme in which the gateway pre-assigns any capable IoT node — parent or non-parent — as the authentication server for newly joined devices. By decoupling authentication from network topology, the scheme distributes the authentication load evenly across the network. Authentication relies on PUF-based secret derivation, a hash-based one-way accumulator for group membership verification, and XOR-masked message exchanges, resulting in an efficient and lightweight protocol.

Despite its efficiency, Das [1] leaves two security gaps unaddressed. First, the real device identity ID_D is transmitted in plaintext on the open channel in every authentication session, enabling an adversary with radio access to track a device across sessions — a direct violation of modern privacy requirements. Second, both the authentication server and the device maintain only a single session state; a packet loss during session key delivery desynchronises their states, causing all subsequent authentication attempts to fail until the device undergoes full re-enrolment, which is an energy-costly operation for battery-powered constrained nodes.

This paper makes the following key contributions.

- 1) An improved scheme over Das [1] is proposed, adding per-session pseudonym rotation for device anonymity and a dual-state desynchronisation recovery mechanism.
- 2) The security of the proposed scheme is validated through both informal analysis and formal verification using ProVerif.
- 3) The efficiency of the proposed scheme is evaluated using both theoretical and simulation-based (COOJA/Contiki-NG) frameworks, emphasizing communication overhead, energy consumption, and CPU utilization in a multihop constrained setting.

Section II elaborates on related literature. Section III presents the proposed scheme. Furthermore, Section IV contains the detailed security analysis. Finally, Section V presents the performance analysis, with Section VI stating the conclusion.

II. LITERATURE SURVEY

Existing schemes for IoT authentication address the problem from multiple directions but each leaves gaps. Distributed approaches reduce relay load but remain topology-bound. Light-

TABLE I
FEATURES COMPARISON OF AUTHENTICATION SCHEMES.

Scheme	Dec.	PUF	Acc.	ASU	Anon.	Desync	C.OH	M.OH
RAAF-MEC [12]	×	✓	×	✓	×	×	H	L
SAKMS [14]	✓	×	×	×	×	×	H	L
Li et al. [13]	✓	✓	×	✓	×	×	H	L
Zheng et al. [11]	✓	✓	×	✓	×	×	M	L
Light-SAE [10]	×	×	×	×	×	×	H	H
Proposed	✓	✓	✓	✓	✓	✓	L	L

Dec.: Decoupled design. *Acc.*: Hash-based accumulator. *ASU*: Auth. secret update. *Anon.*: Device anonymity via pseudonym. *Desync*: Desync. recovery. *C.OH*: Computation — L (Hash/XOR/≤ 2 AES), M (> 2 AES), H (ECC/Pairings). *M.OH*: Message exchanges — L (1–3), M (4–5), H (> 5).

SAE [10] authorise parent nodes as authentication servers, yet a popular parent is overwhelmed by simultaneous join requests from many children. Zheng et al. [11] designed a peer-to-peer PUF-based scheme that lacks a multihop decoupled design. RAAF-MEC [12] deploy multiple PUF-equipped edge servers, but the absence of a genuine multihop topology means certain servers still receive disproportionate load. PUF-based schemes avoid long-term secret storage but trade in overhead. Li et al. [13] and SAKMS [14] combine PUF with session key exchange at the cost of ECC operations, while He et al. [8] achieve lightweight anonymous authentication using hash-and-XOR primitives for CoAP/OSCORE but rely on a centralized server.

Identity-hiding approaches introduce anonymity at prohibitive cost. Al-Riyami and Paterson [15] propose ECC-based certificateless cryptography that is too heavy for constrained nodes, and blockchain-based schemes [16] achieve decentralisation but incur prohibitive storage and consensus overhead. Zhong et al. [17] present a systematic taxonomy of lightweight anonymous IoT authentication, deconstructing existing protocols into nine sub-frameworks (SF1-UA through SF9-FS) and two general frameworks (GF1, GF2), and their analysis demonstrates that only GF2 simultaneously achieves user anonymity, forward secrecy, and desynchronization resistance, a combination absent from all prior lightweight schemes. The proposed scheme follows the GF2 architectural pattern, where the dual-state pseudonym mechanism corresponds to the SF6-UA sub-framework [17], and per-session key derivation from a PUF-assisted fresh nonce provides forward secrecy analogous to GF2. Unlike GF2, which uses a one-time hash chain $hk = h(hk)$, the proposed scheme replaces the hash chain with PUF-derived session randomness and augments membership verification with a hash-based accumulator — mechanisms not modelled in the Zhong et al. taxonomy. Table I contrasts key features across representative schemes. Existing lightweight schemes trade off at least one of these properties, namely decoupled design, PUF, accumulator-based membership, authentication secret update (ASU), anonymity, or desynchronization recovery, whereas the proposed scheme is the first to satisfy all six simultaneously with low computational and communication overhead.

TABLE II
NOTATION.

Symbol	Description
D, AS	Newly joined device and its authentication server
GW	Gateway / root node
$H(\cdot)$	Collision-resistant hash function (SHA-256)
$SE(\cdot), SD(\cdot)$	Symmetric encryption/decryption (AES-128)
$\oplus$	Bitwise XOR operation
$\&$	Bitwise AND operation
c_D, c_{AS-D}	PUF challenges for D (issued by AS) and AS (by D)
m_D	Session-based random issued by AS during enrolment
m_{curr}, m_{old}	Current and previous session-based randoms at D and AS
y_D, R_D	Group secret and PUF response of D
Y_D^H	Hashed unique secret: $H(y_D)$
T_{Acc}	Accumulated group-membership value at AS
Φ_{AS-D}	PUF-response binding: $PUF_{AS}(c_{AS-D}) \oplus R_D$
PID_{curr}, PID_{old}	Current and previous pseudonyms of D
K_{GW-D}	Session key between GW and D
K_{GW-AS}	Long-term shared key between GW and AS
ts_1, ts_2, ts_{auth}	Timestamps for freshness verification

III. PROPOSED AUTHENTICATION SCHEME

A. System Model and Assumptions

The proposed scheme is designed to help nodes establish secure multihop communications with the gateway. To establish a secure communication, nodes should be authenticated by a trusted authentication server. A set of existing IoT nodes inside the network is entrusted with the responsibility of authentication by the gateway in a distributed manner. The gateway also shares a secret key with each of those responsible IoT nodes/authentication servers. The authentication server can be a parent or a non-parent node—they may be *decoupled*—which prevents newly joined child nodes from overwhelming their parent with biased authentication load.

IoT nodes are assumed to be embedded with a PUF and are capable of performing various cryptographic operations, such as encryption/decryption, XOR, hashing, bitwise AND, and concatenation.

Nodes D and AS are assumed to be the newly joined node and its authentication server, respectively. GW represents the gateway symbolically. AS may be located at a singlehop or multihop distance from D . It is assumed that GW has already selected the authentication server set and has established a shared secret key with each member of the set. Let $AS' = \{AS_1, AS_2, \dots, AS_n\}$ be the set of authentication servers selected by GW , such that $AS \in AS'$. GW has a shared secret key K_{GW-AS} with AS . It is also assumed that D , whose distinct identifier is ID_D , possesses the address of its authentication server AS along with its unique identity ID_{AS} . Table II elaborates symbols and functions incorporated in the following scheme.

There are mainly three phases — node enrolment, node authentication, and node session key exchange. The following sections elaborate on these phases.

B. Node Enrolment Phase

The main purpose of this phase is to enrol the unique secret of D (i.e., y_D) with the accumulated value (T_{Acc}) of AS . During the authentication phase, D uses y_D again to prove

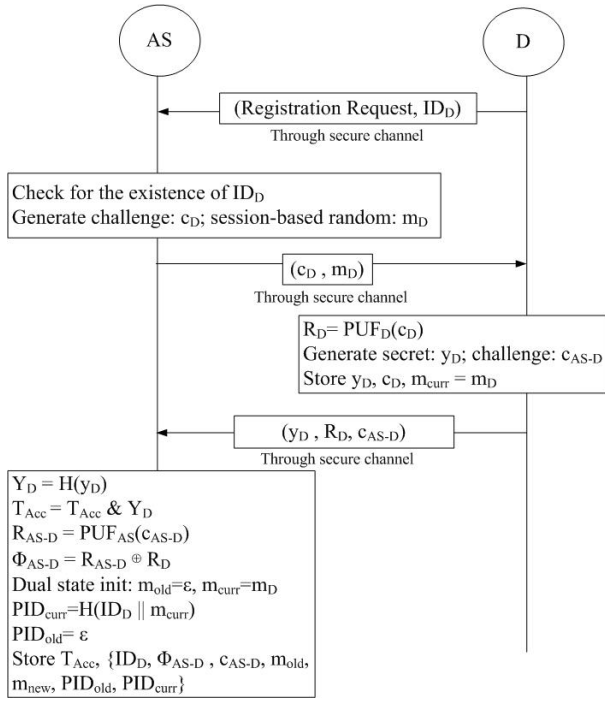


Fig. 1. Phase 1 — Node Enrolment Phase (secure channel).

its group membership. PUF responses and a session-based random are also exchanged to secure the communication of y_D during authentication. Additionally, the phase initialises the pseudonym and dual-state records that protect D 's identity on all subsequent open-channel exchanges. This phase is assumed to be executed once, over an end-to-end encrypted channel, for every newly joined node. Figure 1 details the message flow.

D initiates the phase by sending a registration request along with its unique identity ID_D to AS —the only time ID_D appears in any network message. As a response, AS generates a challenge $c_D \in \mathbb{C}_D$ and a session-based random m_D , and forwards them securely to D . Both credentials secure the communication during authentication. Specifically, c_D enables D to regenerate its PUF response, while m_D introduces unpredictability into open-channel messages, preventing adversarial prediction. With c_D as input to its embedded PUF, D obtains $R_D = \text{PUF}_D(c_D)$. D also generates its unique secret y_D (a random value), and the security of the entire scheme critically depends on maintaining the confidentiality of y_D . D picks a reverse challenge c_{AS-D} and sends (y_D, R_D, c_{AS-D}) back to AS over the secure channel, maintaining the confidentiality of y_D . D stores $\{y_D, c_D, m_{\text{curr}} = m_D\}$.

AS inputs y_D to the hash function $H(\cdot)$, generating the tag $Y_D^H = H(y_D)$, then performs a bitwise AND with the current accumulated value, obtaining $T_{\text{Acc}} = T_{\text{Acc}} \& Y_D^H$. The accumulated value contains the compact, hashed, unique secrets of all enrolled nodes, which are authenticated by AS . In this way, D successfully enrolls itself with AS . AS then generates $R_{AS-D} = \text{PUF}_{AS}(c_{AS-D})$ and calculates $\Phi_{AS-D} = R_{AS-D} \oplus R_D$, keeping R_D in XOR-protected form for further

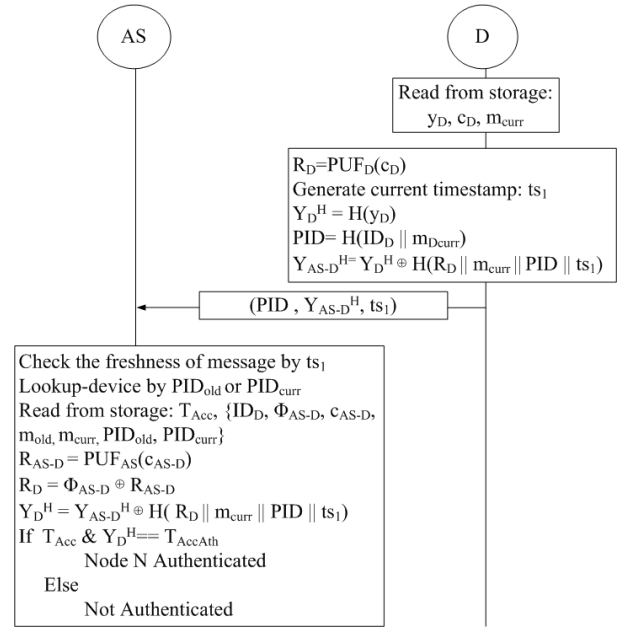


Fig. 2. Phase 2 — Node Authentication Phase (open channel).

secure communications.

To protect D 's identity in all subsequent open-channel messages, AS initialises a per-session pseudonym record. Since ID_D must not appear on the open channel after enrolment, D will instead be identified by the *pseudonym* $PID = H(ID_D || m_{\text{curr}})$ in all future authentication messages. Because H is one-way and m_{curr} is private, this pseudonym cannot be linked to ID_D or to pseudonyms from other sessions by a channel observer. AS precomputes $PID_{\text{curr}} = H(ID_D || m_{\text{curr}})$ and sets $PID_{\text{old}} = \perp$, $m_{\text{old}} = \perp$. The *dual state* (m_{curr} , m_{old} , PID_{curr} , PID_{old}) is maintained because Phase 3 delivers the updated session random m_{new} over the open channel, where a single packet loss can cause D to retain m_{curr} while AS has already advanced. By retaining both the current and the previous pseudonym, AS can recover from such desynchronisation without requiring D to re-enrol. AS stores $\{T_{\text{Acc}}, ID_D, \Phi_{AS-D}, c_{AS-D}, m_{\text{curr}}, m_{\text{old}}, PID_{\text{curr}}, PID_{\text{old}}\}$.

C. Node Authentication Phase

The main purpose of this phase is to prove the membership of D 's unique secret y_D to AS . Since authentication is executed over the open channel, y_D must be communicated in a secure form. Both the PUF-based challenge-response, producing the secret R_D , and the session-based random m_{curr} , incorporating unpredictability, are known only to D and AS . Additionally, the proposed scheme replaces the real identity ID_D with the current pseudonym PID , so that a passive observer cannot link successive sessions to the same device. Figure 2 illustrates the phase.

The phase is initiated by D . D regenerates $R_D = \text{PUF}_D(c_D)$ from the stored challenge c_D , obtained during enrolment. Rather than including its real identity ID_D —which

would enable cross-session tracking— D computes its current pseudonym $PID = H(ID_D \| m_{curr})$. D then produces the hashed secret $Y_D^H = H(y_D)$ in XOR-masked form as $Y_{AS-D}^H = H(y_D) \oplus H(R_D \| m_{curr} \| PID \| ts_1)$. The XOR mask is the hash of R_D (regenerated from c_D), the session-based random m_{curr} , the pseudonym PID , and the current timestamp ts_1 . The timestamp provides replay protection to the XOR-masked Y_D^H . The complete message (PID, Y_{AS-D}^H, ts_1) is sent to AS . An attacker cannot obtain $H(y_D)$ from Y_{AS-D}^H without knowing R_D (which can only be generated by D 's PUF) and m_{curr} (known only to D and AS).

After receiving the message, AS performs a *dual-state lookup* [17], checking whether the received PID matches PID_{curr} or PID_{old} in its records. A match on PID_{old} indicates that D missed the Phase 3 delivery in the previous session and still holds the prior m_{curr} . In this case, AS uses the stored m_{old} for this verification, recovering from the desynchronisation without requiring re-enrolment. AS verifies the freshness of ts_1 . R'_D is calculated as $\Phi_{AS-D} \oplus PUF_{AS}(c_{AS-D})$, where both Φ_{AS-D} and c_{AS-D} are retrieved from D 's enrolment record. AS regenerates the same XOR mask and un.masks Y_D^H , then performs the bitwise AND check: $T_{Acc}^{new} = T_{Acc} \& Y_D^{H'} \stackrel{?}{=} T_{Acc}$. From the properties of the hash-based one-way accumulator, if an element is combined into an accumulated value using bitwise AND and the same operation is again performed between them, it gives back the same accumulated value. If $T_{Acc}^{new} = T_{Acc}$, it proves that the unique secret of D is already a legitimate member of AS , completing authentication. On success, AS proceeds to Phase 3.

D. Node Session Key Exchange Phase

This phase is executed by AS to inform GW about the successful authentication of the newly joined node D and to mediate the session key between GW and D . The proposed scheme additionally derives a fresh pseudonym PID_{new} for D at the end of each session and updates the dual-state records at both AS and GW , ensuring that successive sessions are unlinkable. The two deliveries to D and to GW are sent simultaneously. Figure 3 shows the phase.

AS uses $auth_token_D$ to deliver the session key K_{GW-D} to GW . K_{GW-D} is computed as $H(R'_D \| m_{new})$, where R'_D is the PUF response of D regenerated by AS during the authentication phase and thus known only to AS and D , and $m_{new} = H(n_1)$ is derived from a fresh random n_1 generated by AS . The computed m_{new} also serves as the updated session-based random for the next authentication session between AS and D , incorporating an update to the authentication secret. In addition, AS derives the session key $K_{GW-D} = H(R'_D \| m_{new})$ and the next pseudonym $PID_{new} = H(ID_D \| m_{new})$. Because PID_{new} is based on the fresh m_{new} , it is computationally unlinkable to the PID used in the current session. The session key, ID_{AS} , ID_D , the next pseudonym PID_{new} , and the authentication completion timestamp ts_{auth} are encrypted under K_{GW-AS} to form $auth_token_D$. The

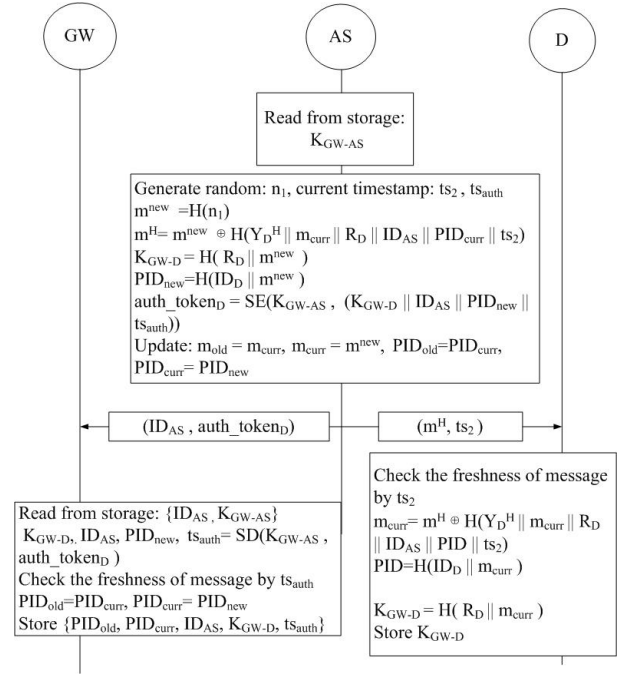


Fig. 3. Phase 3 — Node Session Key Exchange Phase (open channel, simultaneous delivery).

token carries PID_{new} so that GW can update its pseudonym mapping for D 's future communications.

AS also informs D of m_{new} so that D can independently derive K_{GW-D} and its next pseudonym. AS masks m_{new} to protect it from adversaries—if an adversary captures m_{new} , the probability of predicting future session key material increases—so it is masked as $m_H = m_{new} \oplus H(Y_D^{H'} \| m_{curr} \| R'_D \| ID_{AS} \| PID_{curr} \| ts_2)$. Here, $Y_D^{H'}$ is the hashed group secret known to AS from the authentication phase, m_{curr} and R'_D are likewise known from authentication, and ts_2 prevents replay of m_H in a future session. AS simultaneously sends (m_H, ts_2) to D and $(ID_{AS}, auth_token_D)$ to GW . It then saves the current nonce and pseudonym as the previous values and updates its records with m_{new} as the current nonce and PID_{new} as the current pseudonym.

Obtaining (m_H, ts_2) , D first verifies the freshness of ts_2 (freshly generated by AS and included in the delivery), then recovers m_{new} by regenerating the hash mask using its locally held Y_D^H , R_D , m_{curr} , ID_{AS} , PID , and the received ts_2 , and XOR-ing with m_H . D establishes the session key $K_{GW-D} = H(R_D \| m_{new})$, updates its pseudonym $PID_{curr} = H(ID_D \| m_{new})$, updates the stored session nonce to m_{new} , and stores K_{GW-D} .

GW decrypts the token with K_{GW-AS} , verifies ts_{auth} , and extracts K_{GW-D} and PID_{new} . GW saves PID_{curr} as the previous pseudonym, sets PID_{new} as the current one, and stores the session key against both. Retaining PID_{old} ensures that if the Phase 3 delivery to D was lost, GW can still service D 's data frames under the prior session key pending re-authentication.

Algorithm 1 Desynchronization Recovery

Precondition: D holds stale $m_{\text{curr}} = m_{\text{old}}$; AS holds $(m_{\text{curr}}, m_{\text{old}}, PID_{\text{curr}}, PID_{\text{old}})$

Phase 2 — Authentication

- 1: $PID \leftarrow H(ID_D \| m_{\text{curr}}) \triangleright$ stale $m_{\text{curr}} \Rightarrow PID = PID_{\text{old}}$
- 2: $D \rightarrow AS: (PID, Y_{AS-D}^H, ts_1)$
- 3: **if** $PID = PID_{\text{curr}}$ **then**
- 4: AS verifies using m_{curr} $\triangleright$ normal path
- 5: **else if** $PID = PID_{\text{old}}$ **then**
- 6: AS verifies using m_{old} $\triangleright$ desync detected
- 7: **else**
- 8: AS rejects D
- 9: **end if**

Phase 3 — Key Exchange (desync path)

- 10: $m_{\text{new}} \leftarrow \text{rand}()$
 - 11: $m_H \leftarrow m_{\text{new}} \oplus H(Y_D^{H'} \| m_{\text{old}} \| R'_D \| ID_{AS} \| PID_{\text{old}} \| ts_2)$
 $\triangleright AS$ uses $m_{\text{old}}, PID_{\text{old}}$
 - 12: $AS \rightarrow D$ (via GW): (m_H, ts_2)
 - 13: $m_{\text{new}} \leftarrow m_H \oplus H(Y_D^H \| m_{\text{curr}} \| R_D \| ID_{AS} \| PID \| ts_2) \triangleright$
 D uses its own $m_{\text{curr}} = m_{\text{old}}$ — recovers correctly
 - 14: $m_{\text{old}} \leftarrow m_{\text{curr}}; m_{\text{curr}} \leftarrow m_{\text{new}}; PID \leftarrow H(ID_D \| m_{\text{curr}})$
- Result:** D and AS re-synchronised; PID rotated
-

E. Desynchronization Recovery

If the Phase 3 delivery (m_H, ts_2) is lost, D retains its stale m_{curr} , so the pseudonym it computes in the next session equals PID_{old} from AS 's perspective. Algorithm 1 shows how the scheme adapts Phases 2 and 3 to recover without re-enrolment. D 's behaviour is *identical* in both paths—it always uses its own m_{curr} and PID . Only AS adapts: the dual-state lookup detects which nonce D is on, then constructs m_H with the matching $(m_{\text{old}}, PID_{\text{old}})$ so that D 's symmetric decryption succeeds and the states converge. The dual-state mechanism tolerates a single consecutive Phase 3 loss, but two or more consecutive losses exhaust both slots, requiring re-enrolment—a deliberate trade-off between storage overhead and recovery depth.

IV. SECURITY ANALYSIS

A. Formal Security Analysis

The proposed scheme is translated into ProVerif [18] queries under the Dolev–Yao adversary model [19], where the adversary has full control of the public channel and can intercept, replay, modify, and inject messages. Ten queries are verified and all are satisfied. Figure 4 shows the verification summary.

1) *Protocol Correspondence*: Five correspondence queries verify that the protocol steps happen in the correct order and that no adversary can skip or forge a step.

The first query checks enrollment integrity: a device can only be registered at the authentication server after it has explicitly initiated the enrollment process. This rules out

any possibility of a device being enrolled without its own participation.

The second query checks the device side of the authentication round: the device receives the server's acknowledgement and the fresh timestamp ts_2 only after it first sent a valid authentication request. This prevents an adversary from replaying a stale acknowledgement to trick the device into advancing its state.

The third query checks the server side of the same round: the authentication server completes its processing only after it received a structurally correct request from the device. A forged or replayed message cannot push the server into a completed state.

The fourth query binds the two rounds together: the PUF response R_D and the session seed m_D that the device commits in Round 1 are proven to be exactly the same values that appear in Round 2. This rules out an active adversary who might try to substitute credentials between rounds.

The fifth query covers key exchange: the device recovers the new session seed m_{new} only after it legitimately sent the key-exchange request. No key material can be delivered to the device out of sequence.

2) *Secrecy and Forward Secrecy*: Two queries verify the secrecy of the session key $K_{GW-D} = H(R_D \| m_{\text{new}})$ from the perspective of the device and the gateway independently. Even though the adversary observes all traffic on the public channel, it cannot reconstruct K_{GW-D} because the PUF response R_D is never transmitted and m_{new} is always delivered encrypted.

A separate query confirms forward secrecy: the rotated session seed m_{new} itself remains secret. This means that even if an adversary somehow recovers a current session key, it gains nothing about future sessions, because each m_{new} is derived from an independently generated fresh nonce.

3) *Anonymity Preservation*: The most significant secrecy result concerns the real device identity ID_D . After enrollment, ID_D is never sent over the open channel again. Every subsequent message instead carries the pseudonym $PID = H(ID_D \| m_{\text{curr}})$. ProVerif formally confirms — under the full Dolev–Yao active adversary model — that no attacker, even one who intercepts and manipulates every single message on the network, can recover ID_D from the observed traffic.

This result alone proves device anonymity. But anonymity is only meaningful if the pseudonym used in one session cannot be linked to the pseudonym used in the next. The forward-secrecy query addresses this directly: because m_{new} is proven secret, consecutive pseudonyms $H(ID_D \| m_{\text{curr}})$ and $H(ID_D \| m_{\text{new}})$ are computationally indistinguishable to any observer. An adversary who collects pseudonyms from multiple sessions cannot determine whether they belong to the same physical device.

This combination — identity secrecy plus session unlinkability — constitutes a complete machine-verified proof of anonymity. By contrast, LAACA [1] and Zhou [3] rely on informal reasoning or BAN-logic arguments to claim anonymity without a formal model proof under an active adversary.

Verification summary:

```
Query inj-event(DeviceEnrolled(id_D_2,id_AS_3)) ==> inj-event(DeviceEnrollmentStarts(id_D_2,id_AS_3)) is true.
Query inj-event(DeviceAuthenticated(id_D_2,id_AS_3)) ==> inj-event(DeviceAuthenticationStarts(id_D_2,id_AS_3)) is true.
Query inj-event(AuthenticationServerEnds(id_AS_3,id_D_2)) ==> inj-event(AuthenticationServerStarts(id_AS_3,id_D_2)) is true.
Query inj-event(AuthenticationEndsFull(id_D_2,id_AS_3,R_D_2,m_D_2)) ==> inj-event(AuthenticationStartsFull(id_D_2,id_AS_3,R_D_2,m_D_2)) is true.
Query event(DeviceKeyExDone(id_D_2,id_AS_3)) ==> event(DeviceKeyExStarts(id_D_2,id_AS_3)) is true.
Query not attacker(SecretK_GW_D_Device[]) is true.
Query not attacker(SecretK_GW_D_GW[]) is true.
Query not attacker(SecretM_New[]) is true.
Query not attacker(SecretID_D[]) is true.
Weak secret SecretK_GW_D_Device is true.
```

Fig. 4. ProVerif 2.05 verification output — all 10 queries satisfied under the Dolev–Yao active adversary model.

4) *Offline Guessing Resistance*: Finally, a weak-secrecy query confirms that the session key K_{GW-D} cannot be guessed from the public transcript even by an adversary with unlimited offline dictionary power. This closes the gap against brute-force and dictionary attacks that target the key directly from captured packets.

B. Informal Analysis

1) *Replay Attack Resistance*: Phase 1 is executed over a secure channel and immune to replay. Phases 2 and 3 embed timestamps (ts_1 , ts_2 , ts_{auth}) and the session-based random m_{curr} into the hash masks. A replayed message with a stale timestamp or stale m_{curr} will fail the freshness check at AS , D , or GW respectively.

2) *MITM Attack Resistance*: Phase 1 is conducted over an encrypted channel, which precludes MITM. In Phases 2–3, the PUF response R_D (device-specific, unreproducible) and session random m_{curr} (private to D and AS) are embedded in every XOR mask. Without R_D and the current m_{curr} , no adversary can forge a valid masked tag Y_{AS-D}^H or unmask m_H .

3) *Device Anonymity*: ID_D is transmitted only once, over the secure enrolment channel, and is never exposed on the open network again. All open-channel messages use $PID = H(ID_D || m_{curr})$, which is refreshed to $H(ID_D || m_{new})$ after every key exchange. Because m_{new} is derived from a freshly generated nonce n_1 , successive pseudonyms are computationally unlinkable.

4) *Desynchronization Recovery*: If the Phase 3 simultaneous delivery to D fails (packet loss), D retains its m_{curr} unchanged while AS has already advanced to m_{new} . On the next authentication attempt, D computes its PID from the stale m_{curr} , which equals PID_{old} at AS . AS recognizes it via the dual-state lookup and resumes from the consistent prior state without re-enrolment. GW similarly retains the prior pseudonym and its associated session key for continuity.

5) *Forward Secrecy*: $K_{GW-D} = H(R_D || m_{new})$. Compromise of a session key does not reveal previous session keys because m_{new} is independently generated from a fresh nonce and R_D is PUF-derived.

V. PERFORMANCE ANALYSIS

A. Theoretical Performance Analysis

1) *Computational Cost*: The computational cost is a crucial deterministic factor ensuring the security and efficiency of an authentication scheme. Pycrypto, a Python cryptographic library, is used in a Windows system to measure the computational cost of core cryptographic functions, such as SHA-256 (T_{hash}), AES Encryption/Decryption (T_{aes}), and Random generation (T_{rand}). The PUF operation cost (T_{puf}) is taken from prior benchmarks on the same platform [10].

To execute the node enrolment phase (Figure 1):

- AS computes two random generations (T_{rand}), one PUF operation (T_{puf}), and two hash operations (T_{hash}) for pseudonym initialisation.
- On the other hand, D executes two random generations (T_{rand}) and one PUF operation (T_{puf}).

Hence, the total computational cost incurred per node enrolment amounts to $2T_{puf} + 2T_{hash} + 4T_{rand}$. As shown in Table III, the proposed scheme achieves an enrolment cost of 0.18 ms. It competes well with Zhou [3] (0.13 ms) while additionally providing PUF-based secret binding and pseudonym initialisation, both of which are absent from those schemes.

Due to the execution of the authentication and key exchange phase, analysis of its computational efficiency is essential. In the proposed scheme, D and AS (Figures 2 and 3) both execute one PUF operation (T_{puf}).

- D additionally performs six hash operations (T_{hash}): pseudonym computation, group secret hash, and XOR

TABLE III
COMPUTATIONAL COST COMPARISON: ENROLMENT PHASE.

Scheme	Operations	Cost (ms)
LAACA [2]	$3T_{\text{hash}}$	0.11
Zhou [3]	$T_{\text{puf}} + 2T_{\text{hash}} + 3T_{\text{rand}}$	0.13
Zheng et al. [11]	$2T_{\text{puf}} + 8T_{\text{hash}} + 4T_{\text{aes}} + 5T_{\text{rand}}$	0.69
Light-SAE [10]	$4T_{\text{rand}}$	0.01
Proposed	$2T_{\text{puf}} + 2T_{\text{hash}} + 4T_{\text{rand}}$	0.18

Benchmark (avg. 100 iterations): $T_{\text{rand}}=0.0018$ ms, $T_{\text{puf}}=0.0522$ ms, $T_{\text{hash}}=0.0353$ ms, $T_{\text{aes}}=0.0867$ ms.

TABLE IV
COMPUTATIONAL COST COMPARISON: AUTHENTICATION & KEY EXCHANGE PHASE.

Scheme	Operations	Cost (ms)
LAACA [2]	$16T_{\text{hash}}$	0.56
Zhou [3]	$15T_{\text{hash}}$	0.53
Zheng et al. [11]	$2T_{\text{puf}} + 6T_{\text{hash}} + 4T_{\text{aes}} + 5T_{\text{rand}}$	0.67
Light-SAE [10]	$14T_{\text{aes}}$	1.21
Proposed	$2T_{\text{puf}} + 11T_{\text{hash}} + 2T_{\text{aes}} + T_{\text{rand}}$	0.67

mask in Phase 2, followed by mask recovery, session key derivation, and pseudonym rotation in Phase 3.

- *AS* performs five hash operations (T_{hash}), along with one AES encryption (T_{aes}) for the gateway token and one random generation (T_{rand}) for m_{new} .
- *GW* runs one AES decryption (T_{aes}) to recover the session key and updated pseudonym from the forwarded token.

This leads to the total computational cost of the authentication and key exchange phase amounting to $2T_{\text{puf}} + 11T_{\text{hash}} + 2T_{\text{aes}} + T_{\text{rand}}$. In Table IV, the proposed scheme is said to have a computational cost of 0.67 ms. Its computational efficiency surpasses that of the AES-intensive Light-SAE [10] (1.21 ms). Due to the incorporation of a PUF challenge–response pair and an AES-encrypted gateway token — enabling PUF-based group membership verification and decoupled AS selection — the scheme incurs a modest overhead over LAACA [2] (0.56 ms) and Zhou [3] (0.53 ms), which neither employ PUF-based group membership, a decoupled distributed AS, nor dual-state desynchronisation recovery.

2) *Communication Cost*: Communication cost is a core feature for analysing the performance of an authentication scheme, based on the number of message exchanges and their sizes (in bits). By assuming that communications are executed with uniform bit lengths, the communication cost of both enrolment and the authentication and key exchange phase is evaluated and compared with other state-of-the-art schemes.

It is assumed that identities and timestamps are 32 bits, along with nonces, hashes, challenges, responses, and ciphertexts as 256 bits, ensuring a fair comparison.

During the node enrolment phase, *D* enrolls itself with *AS*, executing three message exchanges. The first message communicates the unique identity of *D*, i.e., ID_D (32 bits), to *AS* over the secure channel. In the second message, *AS* transmits a challenge c_D (256 bits) and a session-based random m_D (256 bits) to *D*. The third communication takes place

TABLE V
COMMUNICATION COST COMPARISON: ENROLMENT PHASE.

Scheme	# Messages	Cost (bits)
LAACA [2]	3	2592
Zhou [3]	5	1344
Zheng et al. [11]	3	1600
Light-SAE [10]	2	1056
Proposed	3	1312

IDs/timestamps = 32 b; hashes/nonces/challenges/ciphertexts = 256 b.

TABLE VI
COMMUNICATION COST COMPARISON: AUTH. & KEY EXCHANGE PHASE.

Scheme	# Messages	Cost (bits)
LAACA [2]	3	2400
Zhou [3]	4	2464
Zheng et al. [11]	3	1600
Light-SAE [10]	6	1536
Proposed	3	1376

from *D* to *AS* in the form of a unique secret y_D (256 bits), a PUF response R_D (256 bits), and a reverse challenge c_{AS-D} (256 bits). Thus, the total communication cost per enrolment is 1312 bits as presented in Table V. It outperforms schemes such as LAACA [2] (2592 bits) and Zheng et al. [11] (1600 bits), while remaining competitive with Zhou [3] (1344 bits), which however requires five message rounds.

The authentication and key exchange phase is executed using three message communications. The first message takes place from *D* to *AS*, where *D* communicates its current pseudonym PID (256 bits), a masked membership tag Y_{AS-D}^H (256 bits), and a timestamp ts_1 (32 bits). In the second message, *AS* transmits the masked session seed m_H (256 bits) and a timestamp ts_2 (32 bits) to *D*. The third communication takes place from *AS* to *GW*, consisting of the updated pseudonym PID_{new} (256 bits), the identity ID_{AS} (32 bits), and the encrypted gateway token auth_token_D (256 bits). Thus, the total communication cost becomes 1376 bits, which appears to be the lowest among the compared schemes in Table VI. Das [1] requires 928 bits, with the 448-bit increase directly reflecting the widening of the device identifier from a 32-bit ID_D to a 256-bit pseudonym PID — the direct cost of adding device anonymity to all open-channel communications.

B. Simulation-Based Performance Analysis

A simulation-based performance evaluation has been conducted to detect the efficiency of the proposed authentication scheme using the COOJA simulator within the Contiki-NG framework. Three complementary studies are reported. The first is an overall per-device comparison of the proposed scheme against LAACA [2] and Zhou [3] in a fixed 100-node network comprising 1 gateway, 79 AS nodes (2 active), and 20 newly joined devices. The second is an authentication server (AS) pool-size study that varies the number of active AS nodes from 2 to 10. The third is a network scalability study that grows the total node count from 50 to 100. All results are averaged over five independent random seeds. Table VII elaborates the common simulation parameters.

TABLE VII
SIMULATION PARAMETERS.

Parameter	Value
Simulator	COOJA (Contiki-NG)
Mote type	TelosB (emulated)
Network topology (study i)	100 nodes: 1 GW, 79 AS (2 active), 20 devices
Phases measured	Enrolment, Authentication, Key Exchange
Schemes compared	Proposed scheme, LAAKA [2], Zhou [3]
Active AS (study ii)	2, 5, 10
Network size (study iii)	$N \in \{50, 80, 100\}$ nodes
Random seeds	5 per configuration

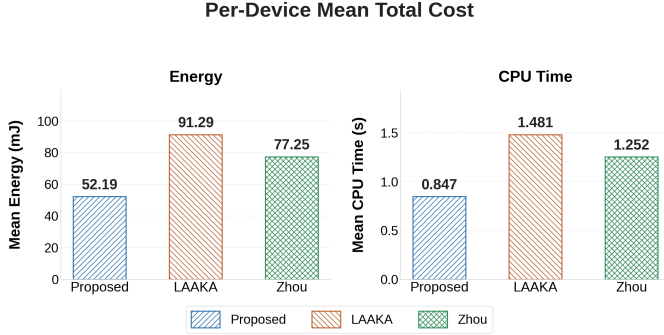


Fig. 5. Per-device mean total cost — all phases: (a) energy and (b) CPU time.

1) Performance Analysis for Overall Per-Device Cost:

Initially, the authentication schemes are evaluated by considering the overhead of one complete authentication cycle — enrolment, authentication, and key exchange — per device. For 20 newly joined nodes in a fixed 100-node network (1 gateway, 79 AS nodes with 2 active, and 20 newly joined devices), the proposed scheme demonstrated a consistent advantage over LAAKA [2] and Zhou [3] by achieving a reduction of 42.8 % and 34.6 % with respect to total per-device energy consumption (Figure 5a), respectively. Similarly, a reduction of 42.7 % and 35.3 % is demonstrated with respect to total per-device CPU time (Figure 5b) over LAAKA and Zhou, respectively. The dominant savings arise in the authentication and key exchange phase, where the proposed scheme consumes 28.87 mJ against 77.97 mJ for LAAKA and 57.06 mJ for Zhou. Due to the incorporation of fewer message exchanges and computationally lightweight hash-XOR operations, the proposed scheme has minimised radio activity and computational load compared to the message-intensive and cryptographically heavier LAAKA and Zhou schemes.

2) *Performance Analysis for Authentication Server Pool Size:* In the decoupled design, the gateway selects any capable node as the authentication server, so the size of the AS pool directly influences load distribution and overall network performance. The performance of the proposed scheme, LAAKA, and Zhou is evaluated by varying the number of active AS nodes from 2 to 10. Considering the total energy consumption (Figure 6a), a reduction of approximately 42 % is demonstrated by the proposed scheme over LAAKA at every AS count. Both schemes improve from AS = 2 to AS = 10, with the proposed

scheme dropping from 1065 mJ to 936 mJ and LAAKA from 1827 mJ to 1631 mJ, suggesting that a pool of 10 AS nodes is near-optimal for a 20-device network. Similarly, as shown in Figure 6(b), a reduction of approximately 42 % in total CPU time is demonstrated over LAAKA across all AS counts. Zhou’s total cost remains flat at approximately 1503 mJ in energy and 24.4 s in CPU time regardless of AS pool size, confirming that its gateway-coupled design is insensitive to AS selection diversity. These consistent reductions demonstrate the energy and computational efficiency of the decoupled AS design in the proposed scheme.

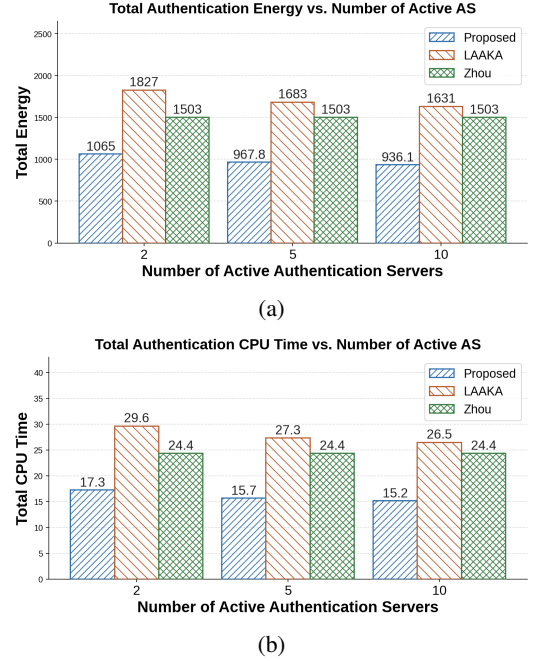


Fig. 6. Total authentication cost vs. number of active AS nodes (20 devices): (a) energy and (b) CPU time.

3) *Performance Analysis for Network Scalability:* The performance of authentication schemes is also analysed by considering their scalability with respect to total network size N , varied from 50 to 100 nodes. Considering the total energy consumption (Figure 7a), a reduction of 42.2 % and 50.6 % is demonstrated by the proposed scheme at $N = 100$ over LAAKA (898 mJ) and Zhou (1051 mJ), respectively. Similarly, as shown in Figure 7(b), a reduction of 42.5 % and 50.6 % is demonstrated with respect to total CPU time over LAAKA (14.6 s) and Zhou (17.0 s), respectively. LAAKA and Zhou exhibit steeper energy and CPU growth with increasing N because their heavier per-device cryptographic load — AES-intensive authentication in LAAKA and a combined heavy Auth+KeyEx round in Zhou — accumulates faster as the device count grows. Due to the incorporation of lightweight XOR-hash operations and a decoupled AS design, the proposed scheme keeps per-device cost low and achieves the lowest total cost among all compared schemes across all measured network sizes. These consistent reductions across all network sizes demonstrate the scalability and computational efficiency

of the proposed scheme in comparison to the benchmark schemes.

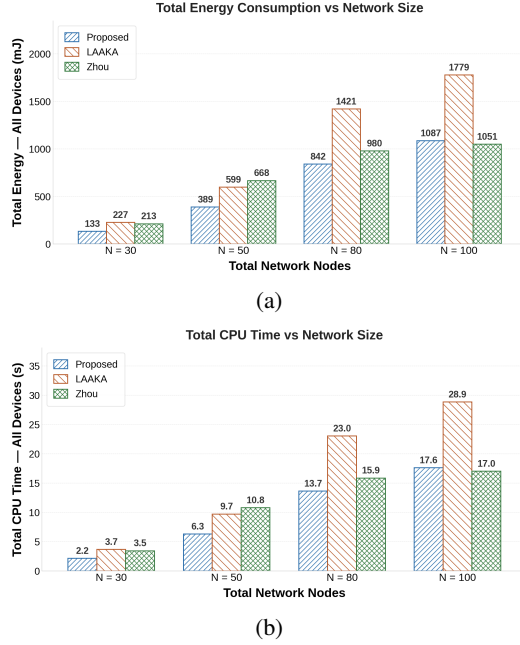


Fig. 7. Network scalability: (a) total energy and (b) total CPU time vs. network size N .

VI. CONCLUSION

This paper presented a lightweight, anonymous, and decoupled distributed authentication scheme for multihop IoT networks. The assigned authentication server—which may or may not be the parent of the newly joined device—performs group-membership verification (Phase 2) and session key establishment (Phase 3) in two clean rounds. Device anonymity is preserved through per-session pseudonym rotation, preventing adversarial tracking across sessions. A dual-state mechanism at both the authentication server and the gateway provides transparent desynchronization recovery without re-enrolment. PUF-based authentication eliminates long-term secret storage, and hash-based accumulator operations with XOR masking keep the computational overhead low.

Formal verification in ProVerif confirms authentication correspondence, session key secrecy, resistance to offline guessing attacks, and observational-equivalence-based anonymity. COOJA/Contiki-NG simulations on emulated TelosB nodes confirm practical suitability. The proposed scheme achieves a per-device three-phase energy of 52.21 mJ, reducing total energy by 42.8 % over LAAKA [2] and 34.6 % over Zhou [3], with consistent gains across varying network sizes (up to $N = 100$ nodes) and AS pool sizes.

REFERENCES

- [1] S. Das and M. Khatua, "Lightweight and decoupled distributed authentication for multihop IoT networks," in *Proc. IEEE COMSNETS*, 2026.
- [2] H. Ali and I. Ahmed, "LAAKA: A lightweight anonymous authentication and key agreement scheme for IoT-fog environment," *Computers & Security*, vol. 140, p. 103770, 2024.
- [3] X. Zhou, S. Wang, K. Wen, B. Hu, X. Tan, and Q. Xie, "Security-enhanced lightweight and anonymity-preserving user authentication scheme for IoT-based healthcare," *IEEE Internet of Things Journal*, vol. 11, no. 6, pp. 9718–9731, 2024.
- [4] I. Zhou, I. Makhdoom, N. Shariati, M. A. Raza, R. Keshavarz, J. Lipman, M. Abolhasan, and A. Jamalipour, "Internet of things 2.0: Concepts, applications, and future directions," *IEEE Access*, vol. 9, pp. 70961–71012, 2021.
- [5] N. Lajnef, A. Mami, and F. Derbel, "Applications of wireless sensor networks and IoT frameworks in industry 4.0: A systematic literature review," *Sensors*, vol. 22, no. 6, 2022.
- [6] M. Khatua, S. Das, and P. M. Rana, "PUF-based lightweight mutual authentication scheme for IoT-based smart grids," in *Proc. IEEE ANTS*. IEEE, 2024, pp. 1–6.
- [7] S. A. Sheik and A. P. Muniyandi, "Secure authentication schemes in cloud computing with glimpse of artificial neural networks: A review," *Cyber Security and Applications*, vol. 1, p. 100002, 2023.
- [8] D. He, Y. Cai, S. Zhu, Z. Zhao, S. Chan, and M. Guizani, "A lightweight authentication and key exchange protocol with anonymity for IoT," *IEEE Trans. Wireless Communications*, vol. 22, no. 11, pp. 7862–7872, 2023.
- [9] S. Roy, M. H. Mahalat, and B. Sen, "Design and implementation of cost-effective end-to-end authentication protocol for PUF-enabled IoT devices," *IEEE Trans. Emerging Topics in Computing*, 2025.
- [10] P. Rosa, A. Souto, and J. Cecilio, "Light-SAE: A lightweight authentication protocol for large-scale IoT environments made with constrained devices," *IEEE Trans. Network and Service Management*, vol. 20, no. 3, pp. 2428–2441, 2023.
- [11] Y. Zheng, W. Liu, C. Gu, and C.-H. Chang, "PUF-based mutual authentication and key exchange protocol for peer-to-peer IoT applications," *IEEE Trans. Dependable and Secure Computing*, vol. 20, no. 4, pp. 3299–3316, 2022.
- [12] O. Alruwaili, M. Tanveer, S. A. Aldossari, S. Alanazi, and A. Armghan, "RAAF-MEC: Reliable and anonymous authentication framework for IoT-enabled mobile edge computing environment," *Internet of Things*, vol. 29, p. 101459, 2025.
- [13] S. Li, T. Zhang, B. Yu, and K. He, "A provably secure and practical PUF-based end-to-end mutual authentication and key exchange protocol for IoT," *IEEE Sensors Journal*, vol. 21, no. 4, pp. 5487–5501, 2020.
- [14] W. Yang, C. Hou, Y. Wang, Z. Zhang, X. Wang, and Y. Cao, "SAKMS: A secure authentication and key management scheme for IETF 6TiSCH industrial wireless networks based on improved elliptic-curve cryptography," *IEEE Trans. Network Science and Engineering*, vol. 11, no. 3, pp. 3174–3188, 2024.
- [15] S. Al-Riyami and K. G. Paterson, "Certificateless public key cryptography," in *Proc. ASIACRYPT*, ser. LNCS, vol. 2894. Springer, 2003, pp. 452–473.
- [16] M. T. Al Ahmed, F. Hashim, S. J. Hashim, and A. Abdullah, "Authentication-chains: Blockchain-inspired lightweight authentication protocol for IoT networks," *Electronics*, vol. 12, no. 4, p. 867, 2023.
- [17] J. Zhong, S. He, Z. Liu, and L. Xiong, "Lightweight anonymous authentication for IoT: A taxonomy and survey of security frameworks," *Sensors*, vol. 25, no. 17, p. 5594, 2025.
- [18] B. Blanchet, B. Smyth, V. Cheval, and M. Sylvestre, "ProVerif 2.00: Automatic cryptographic protocol verifier, user manual and tutorial," INRIA, Tech. Rep., 2018.
- [19] D. Dolev and A. Yao, "On the security of public key protocols," *IEEE Trans. Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.